



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #4
Searching on the file data using array of pointers***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Searching on the file data using array of pointers**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

When working with files in C, efficient data handling is crucial, especially when the file contains multiple entries such as lines, records, or words. One powerful technique is using an **array of pointers** to store and access the data dynamically. This approach **allows flexible memory management, faster search operations, and easier data manipulation without altering the original file content**. By storing each line or record in a dynamically allocated memory block and keeping track of these blocks through an array of pointers, we can perform searches or any modifications in-memory, leading to optimized and structured file processing.

Let us consider a simple text file called numbers.txt which contains many integers in an ascending order, separated by any white space in between. The file is well structured and very well formatted. The requirement is to search for the given integer in the file and display appropriate message.

Coding Example_1: Recursive binary search on an array having the integers from the file.

```
#include <stdio.h>
#include <stdlib.h>
int Search(int **aop, int low, int high, int key);
int main(){
    FILE* fp;
    fp = fopen("numbers.txt", "r");
    if(fp == NULL)
        printf("Can't open the file\n");
    else {
        int a;          int n = 0;
        while(fscanf(fp, "%d", &a) != -1)
            n++;
        int *p = malloc(n*sizeof(int));
        int **aop = (int **)malloc(n * sizeof(int *));
        for(int i = 0; i < n; i++)
            aop[i] = &p[i];
        rewind(fp);
        int i = 0;
        while(fscanf(fp, "%d", &p[i]) != -1)
            i++;
        fclose(fp);
```

```

printf("\nDisplaying the contents of the dynamic array using array of integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", p[i]);
printf("\nDisplaying the contents of the dynamic array using array of pointers to
integers\n");
for(int i = 0 ; i < n; i++)
    printf("%d\t", *aop[i]);
printf("\nEnter the integer to be searched\n");
int key_int;
scanf("%d", &key_int);
int res = Search(aop,0, n, key_int);
if(res == -1)
    printf("The key integer doesn't exist in the file\n");
else
    printf("Found at position %d", res);
}
return 0;
}
int Search(int **aop, int low, int high, int key){
    if(low > high) // base condition
        return -1;
    else{
        int mid = (low+high)/2;
        if(*aop[mid] == key)
            return mid;
        else if(key < *aop[mid])
            return Search(aop,low,mid-1,key);
        else
            return Search(aop,mid+1,high,key);
    }
}

```

```

Displaying the contents of the dynamic array using array of integers
10    20    30    67    69    74    80    84    95    100
    234    345    567
Displaying the contents of the dynamic array using array of pointers to integers
10    20    30    67    69    74    80    84    95    100
    234    345    567
Enter the integer to be searched
89
The key integer doesn't exist in the file
C:\Users\Dell>a

Displaying the contents of the dynamic array using array of integers
10    20    30    67    69    74    80    84    95    100
    234    345    567
Displaying the contents of the dynamic array using array of pointers to integers
10    20    30    67    69    74    80    84    95    100
    234    345    567
Enter the integer to be searched
30
Found at position 2

```

Now let us consider the array of structures and array of pointers to array of structures. Given data file - > **student_rollno_name.csv**. If the roll_number of the student is given, display the details of that student.

Data file Description:

- Contains the information of students in two columns only.
- First row represents the column headers such as roll_number and name.
- Contains around 55 rows. Every data is stored with a comma in between.

Coding Example_2: Searching the details of student based on roll_number

```
typedef struct student
{
    int roll;      char name[20];  }STUDENT;
void init_ptr(struct student s[], struct student *p[], int n);
void disp(struct student* p[], int n) ;
int Search(STUDENT **aop, int low, int high, int key);
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main(){
    FILE *fr=fopen("sorted_data.csv","r");
    if(fr == NULL) {
        printf("Cant access the file\n");exit(1);
    }
    char a[200];  fgets(a,200,fr);      char *item; int i=0;
    struct student s[100];              // Array of structures
    // Reading a line from the file and splitting it based on the delimiter comma(,), copying the details
    to array of student structure
    while(fgets(a,200,fr)) {
        item=strtok(a,",");      s[i].roll=atoi(item);
        item=strtok(NULL,",");   strcpy(s[i].name,item);
        i++;
    }
    int n = i;
    fclose(fr);
    struct student *p[100]; //Creation of array of pointers to point to array of structures
    init_ptr(s, p, n);  // every element of array of pointer is made to point to every element of
    array of structure
    disp(p,n);          // to display the array of structures using array of pointers
    int key_rollno;
    printf("Enter the roll number of the student to be searched\n");
    scanf("%d",&key_rollno);
    int res = Search(p, 0, n-1, key_rollno);
    if(res == -1)
        printf("Student not found\n");
    else{
        printf("Student details are here\n");
        printf("Roll_number %d, Name is %s",p[res]->roll, p[res]->name);
    }
    return 0;
}
```

```
int Search(STUDENT **aop, int low, int high, int key){
    if(low > high) // base condition
        return -1;
    else {
        int mid = (low+high)/2;
        if(aop[mid]->roll == key)
            return mid;
        else if(key < aop[mid]->roll)
            return Search(aop,low,mid-1,key);
        else
            return Search(aop,mid+1,high,key);
    }
}

void init_ptr(struct student s[], struct student *p[], int n){
    for(int i = 0 ; i < n; i++)
        p[i] = &s[i];
}

void disp(struct student* p[], int n) {
    for(int i = 0 ; i < n; i++)
        printf("%d %s", p[i]->roll, p[i]->name);
}
```

```
Enter the roll number of the student to be searched
105
Student not found
```

```
Enter the roll number of the student to be searched
43
Student details are here
```

Think about this-> How to compare the strings if user wants to enter the student name and get other details of the student?

Happy Searching file contents using Array of Pointers!!